We now must create the read and write components of the stream. The read stream reads in the Delta table we created in *Chapter 3*. Recall we created the Bronze layer by streaming in JSON files and writing them to our `inputTable`. The `readStream` and `writeStream` code is long so we will break it up into smaller sections in the following steps:

1. You should recognize this from earlier chapters. The difference is we are using `selectExpr` to isolate `CustomerID` and `TransactionTimestamp`. Additionally, we specifically set the type of output dataframe to the case class that `flatMapGroupsWithState` is expecting:

```
val inputDf =
  spark.readStream
    .format("delta")
    .schema(schema)
    .table(inputTable)
    .selectExpr("CustomerID", "cast(TransactionTimestamp as timestamp) TransactionTimestamp")
    .as[InputRow]
```

Figure 5.22 – InputDF is a read stream reading the table we created in Chapter 3

2. We apply the watermark and `flatMapGroupsWithState` function to `inputDf`. In Spark Streaming, a watermark is a time threshold determining the maximum allowable delay for late events. We're allowing data to be 30 seconds late before it is dropped (*Watermarking in Spark Structured Streaming*, by Thomas Treml: `https://towardsdatascience.com/watermarking-in-spark-structured-streaming-9e164f373e9`). The `flatMapGroupsWithState` function is an arbitrary stateful streaming aggregation operator. It applies our `updateState` function to each micro-batch of transactions while maintaining the state for each customer ID:

```
val flatMapGroupsWithStateResultDf =
  inputDf
    .withWatermark("TransactionTimestamp", "30 seconds")
    .groupByKey(_.CustomerID)
    .flatMapGroupsWithState(OutputMode.Append,
        GroupStateTimeout.EventTimeTimeout)(updateState)
```

Figure 5.23 – Applying the watermark and flatMapGroupsWithState function to inputDf

3. We define the `updateCounts` function for `foreachBatch` to update the counts in the write stream. It performs upserts of the new transaction counts into the `transaction_count_ft` table; this is the CDC component: